Event-Driven Programming

Tkinter and a web page, same logic

Computer Science II • Dr. Jeremy P. Evert

October 19, 2026

Today's destination

TODAY'S DESTINATION

- Map an event: source, handler, state, output.
- Trace it from the event to the refreshed display.
- Test the rule with no UI.
- Verify the handler only routes.
- Swap the front end without touching the model.

▶ *Cue: Say it out loud: the front end is a vehicle. The model is what we keep.*

What is an event-driven program?

CORE CONCEPT

Cycle	Something happens → a handler runs → state changes → the display refreshes.
Examples	A key press, a timer, a network packet, a button click, a web request.

- *Cue: Ask: which of these have you already written a handler for?*

Cousins beyond GUIs

CORE CONCEPT

Hardware interrupts A device signals; the CPU runs a handler. That handler is a callback.

State machines FPGA and digital designs often react to input signals and change state.

- *Cue: Say "cousins." Same shape of idea, different machinery.*

Map the event: Inventory restock

Event	Restock is requested
Source	Button click (Tkinter) or POST /restock (web)
Handler	`on_restock` or `do_POST`
State	The model's `items` list
Output	"Items: n" shown to the user

- *Cue: Have students fill in this table for their own world noun first.*

The model stays the same

```python
# inventory.py
class Inventory:
    def __init__(self):
        self.items = []

    def restock(self, name):
        self.items.append(name)
        return len(self.items)

    def count(self):
        return len(self.items)
```

```python
# test_inventory.py
class InventoryTest(unittest.
    TestCase):
    def test_restock_adds_one(self
        ):
        inv = Inventory()
        self.assertEqual(inv.
            restock("rope"), 1)
        self.assertEqual(inv.count
            (), 1)

    def test_three_restocks(self):
        inv = Inventory()
        for _ in range(3):
            inv.restock("rope")
        self.assertEqual(inv.count
            (), 3)
```

Pass 1: the Tkinter view

PREDICT BEFORE RUNNING

```python
import tkinter as tk
from inventory import Inventory

model, root = Inventory(), tk.Tk()
text = tk.StringVar(value="Items: 0")
tk.Label(root, textvariable=text).pack()

def on_restock():
    n = model.restock("rope")    # route to the model
    text.set(f"Items: {n}")      # then refresh the view

tk.Button(root, text="Restock", command=on_restock).pack()
root.mainloop()
```

Predict: three clicks on Restock. What does the label say?

Pass 2: a web page view

```
model = Inventory()

PAGE = """<!doctype html><title>Inventory</title>
<p>Items: {n}</p>
<form method="post" action="/restock"><button>Restock</button></form>"""

class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        if self.path == "/restock":
            n = model.restock("rope")      # route to the model
            print("request -> handler -> model.restock -> {n} items")
            self.send_response(303)        # then refresh the view
            self.send_header("Location", "/")
            self.end_headers()
        else:
            self.send_error(404)
```

Same Inventory. Python standard library only (`http.server`); run `python3 view_web.py`, open `localhost:8000`.

Same logic, two front ends

	Tkinter	Web page
Event source	Button click	POST /restock
Handler wiring	`command=on_restock`	`do_POST`
Display	`text.set` on a label	Page shows the count
Unchanged	`Inventory`, its test, the restock rule	

- *Cue: Ask: what did we have to edit in inventory.py to add the web page? (Nothing.)*

Trace the event in each

CODE EXAMPLE

Tkinter: click → callback → model.restock → 1 → refresh

Web: request → handler → model.restock → 1 → response

```
request -> handler -> model.restock -> 1 items
```

- *Cue: Students paste their own trace into the World Bible as evidence.*

Debug and verify: break the boundary

COMPARE TWO APPROACHES

Boundary broken

```
items = []    # view owns data
def on_restock():
    items.append("rope")
    n = len(items)
    text.set(f"Items: {n}")
```

Boundary kept

```
model = Inventory()
def on_restock():
    n = model.restock("rope")
    text.set(f"Items: {n}")
```

- *Cue: If you cannot test a rule without the UI, the rule is in the wrong place.*

Dependencies are an avenue, not the point

- Tkinter needs `python3-tk` on Ubuntu.
- The web version needs nothing beyond Python.
- Setup trouble is incidental. The model and its test run either way.

▶ *Cue: If a laptop will not open a window, switch to the web pass and keep going.*

Peer swap and World Bible

DESIGN DECISION

Contract test: trade model files with a partner, then run `python3 contract_test.py their_module`.

- It prints "contract holds": your front end works on their model.
- It fails: decide whose method names or return values should change.

World Bible entry: the rule you moved out of a handler, and your trace.

Recap

RECAP

- An event runs a handler, the handler changes state, the display refreshes.
- The model and its test do not change when the front end does.
- Handlers route; rules live in the model.
- Tkinter and a web page are two vehicles for one design.

Exit question

EXIT QUESTION

Which part of your program would you keep if the front end changed tomorrow, and why?

Gate: due Friday 11:59 PM Central.